

Mock Interviews

Start new sessions with /api/interviews/sessions/start.

company slug (optional)

DSA_CODING

DSA_THEORY

OS

DBMS

NETWORKS

SYSTEM_DESIGN

TOTAL QUESTIONS

5

Start Interview

Sessions

- Session #11

0/5 answered • COMPLETED
- Session #10

0/5 answered • COMPLETED
- Session #9

0/5 answered • COMPLETED
- Session #8

0/2 answered • COMPLETED
- Session #7

0/4 answered • COMPLETED
- Session #6

0/4 answered • COMPLETED
- Session #5

1/5 answered • COMPLETED
- Session #4

0/5 answered • COMPLETED
- Session #3

0/5 answered • COMPLETED
- Session #2

2/5 answered • COMPLETED

Company: General pool
Status: COMPLETED
Score: 0.00 / 10

QUESTION 5 • DBMS

Design a data structure to store a large number of intervals on a line, and implement a function to find all intervals that overlap with a given interval. The intervals are in the form of [start, end] where start <= end. The function should return a list of overlapping intervals.

Question Skipped
You chose to skip this question. Fetch the next question to continue.

Question skipped. Fetch next question to continue.

Session Review & Evaluation

Refresh Status

Question 1 DSA_CODING MEDIUM

Grade: 6/10

QUESTION PROMPT

Write a program to implement a LFU (Least Frequently Used) cache. The cache should have a capacity and support get and put operations. When the cache is full, the least frequently used item should be evicted. Sample test case: Create an LFU cache with capacity 2, put(1, 1), put(2, 2), get(1), put(3, 3), get(2), Expected output: 1, -1

YOUR ANSWER

```
#include <bits/stdc++.h>

using namespace std;

typedef long long ll;

class LFU
{
public:
    unordered_map<int,int> m;
    int capacity;
    LFU(int cap){
        capacity=cap;
    }
    void put(int x,int y){
        if(m.size()==capacity)
        {

        }
    }
}
```

EVALUATION & FEEDBACK

Verdict: Needs Work
Feedback: The candidate correctly identifies clock speed and the number of cores as factors influencing CPU performance but misses cache size, a critical factor. The answer is partially correct but incomplete.

REFERENCE ANSWER

LFU Cache Implementation

Problem Understanding

The problem requires implementing a Least Frequently Used (LFU) cache with a given capacity. The cache should support two operations: `get` and `put`. The `get` operation retrieves the value associated with a given key if it exists in the cache; otherwise, it returns -1. The `put` operation inserts or updates the value associated with a given key. When the cache is full, the least frequently used item should be evicted.

Key constraints and edge cases:

- The cache has a limited capacity.

Question 2 DSA_CODING MEDIUM

Grade: 6/10

QUESTION PROMPT

Given a string and an integer k, find the length of the longest substrng with at most k distinct characters. For example, for the string 'eceba' and k = 2, the expected output is 3

because 'ece' is the longest substring with at most 2 distinct characters.

YOUR ANSWER

```
#include <bits/stdc++.h>
using namespace std;

typedef long long ll;

int main() {
    string x;
    int k;
    cin >> x;
    cin >> k;
    unordered_map<char,int> m;
    int ans=0,l=0;
    for(int i=0;i<s.size();i++)
    {
        m[s[i]]++;
        while(m.size()>k)
        {
            m[s[l]]--;
            if(m[s[l]]==0)
                m.erase(s[l]);
            l++;
        }
        ans=max(ans,i-l+1);
    }
    return ans;
}
```

EVALUATION & FEEDBACK

Verdict: Needs Work

Feedback: The candidate correctly identifies clock speed and the number of cores as factors influencing CPU performance but misses cache size, a critical factor. Their explanation shows a basic understanding but lacks completeness.

REFERENCE ANSWER

Problem Understanding

The problem requires finding the length of the longest substring with at most k distinct characters in a given string. The input consists of a string `s` and an integer `k`, and the output is the length of the longest substring with at most `k` distinct characters.

Key constraints and edge cases:

- The string `s` can be empty.
- The integer `k` can be 0.
- The string `s` can have only one character.

Question 3 DSA_THEORY MEDIUM

Skipped

QUESTION PROMPT

Tell me about a time when you had to implement something challenging. Please provide a specific example and walk me through your thought process, the steps you took, and the outcome.

REFERENCE ANSWER

1. Direct Answer

I recall a particularly challenging project where I had to implement a scalable and fault-tolerant distributed caching system for a high-traffic e-commerce platform. The goal was to reduce database load and improve page load times.

2. Brief Explanation

The task involved designing and implementing a caching layer that could handle a large volume of requests, ensure data consistency, and provide a high degree of availability. The system had to be integrated with the existing architecture, which included a load balancer, multiple web servers, and a relational database.

3. Real-world Example or Analogy

To tackle this challenge, I drew an analogy with a high-performance, automated library system. Just as a library system needs to efficiently manage book inventory, retrieve

Question 4 DSA_THEORY MEDIUM

Skipped

QUESTION PROMPT

Tell me about a time when you missed a deadline. What were the circumstances, and what did you learn from the experience?

REFERENCE ANSWER

Direct Answer: I recall a situation where I missed a deadline for a critical software delivery. It was during a project where I was the lead engineer, and we were tasked with integrating a third-party API with our existing system.

Brief Explanation: The deadline was missed due to unforeseen complexities in the API integration, which required additional time to resolve. Specifically, the third-party API had limited documentation, and our team encountered unexpected issues with authentication and data mapping. Despite working diligently, I underestimated the time required to overcome these challenges, resulting in a one-week delay.

Real-world Example or Analogy: This experience is similar to planning a road trip with an unexpected detour. You plan your route, but unforeseen road closures or construction can force you to take a detour, adding extra time to your journey. In this case, the detour was the complexity of the API integration, which required us to take a different approach and allocate more time to complete the task.

Question 5 DBMS

MEDIUM

Skipped

QUESTION PROMPT

Design a data structure to store a large number of intervals on a line, and implement a function to find all intervals that overlap with a given interval. The intervals are in the form of $[start, end]$ where $start \leq end$. The function should return a list of overlapping intervals.

REFERENCE ANSWER

To solve this problem, we can use an Interval Tree data structure. An Interval Tree is a type of binary search tree where each node represents an interval. Each node will have a start and end value representing the interval, and a pointer to its left and right children.

Here is a step-by-step approach:

1. Create a class `IntervalTreeNode` to represent each node in the tree. Each node will have 'start', 'end', 'max_end' (the maximum end value in the subtree rooted at this node), 'left', and 'right' attributes.
1. Create a class `IntervalTree` with a method to insert intervals into the tree and a method to find overlapping intervals.
1. When inserting an interval, we start at the root and recursively traverse the tree. At each node, if the interval's start is less than the node's start, we go to the left subtree.